

RIZAL_v1: Project Evolution Documentation

Overview

This document presents a comprehensive technical overview of the modifications applied to the RIZAL_v1 repository following its initial clone. The project underwent four major phases of development, each aimed at improving user interface consistency, enhancing system functionality, strengthening security protocols, and improving overall reliability.

These enhancements collectively modernized the system while maintaining scalability and maintainability.

Phase 1: "Aura" Dark Mode Modernization

Objective

The primary objective of this phase was to achieve 100% visual consistency for the Aura Dark Mode across all administrative and student interfaces.

Architectural Changes

Semantic Theming Implementation

More than 50+ hardcoded hex and RGB color values were replaced with CSS variables to enable centralized theme management.

This change improved:

- **Maintainability**
- **Scalability**
- **UI consistency**

Theme variables were defined in:

`src/config/theme.js`

Component Refactoring

Major UI components were refactored to adopt the Aura Glass Design System, including:

- **aura-glass-card**
- **aura-mesh-bg**

These components introduced:

- Glassmorphism effects
- Consistent dark theme styling
- Premium UI appearance

A total of 11 major views were updated.

Unified Header Architecture

Previously, separate headers existed for:

- Student
- Admin
- School IT

These were consolidated into a single reusable component:

StandardHeader.vue

This header dynamically adapts based on user roles.

Benefits:

- Reduced code duplication
 - Improved maintainability
 - Consistent UI behavior
-

Key Files Modified

src/config/theme.js

src/assets/css/main.css

src/views/desktop/dashboard/SchoolIt*.vue

src/views/desktop/dashboard/SgDashboardView.vue

Phase 2: Functional Dashboard Interactivity

Objective

Transform the dashboard from a static UI prototype into a fully interactive web application using a session-based mock engine.

Implementation Details

Local State Management

A reactive local data store was implemented:

src/stores/useSchoolItPreviewStore.js

This store simulates backend database functionality inside the browser session.

Capabilities include:

- **Data persistence during session**
 - **Real-time UI updates**
 - **Mock backend simulation**
-

Interactive CRUD Features

Schedule View

Users can now:

- **Create events**
- **Edit events**
- **Delete events**

All changes update dynamically within the dashboard.

Student Assignment System

Unassigned students can now be:

- **Assigned to events**
- **Updated in real-time**

This directly updates:

- **Attendance Monitor**
- **Event tracking system**

Settings Interactivity

Settings panel forms were connected to the reactive store, allowing:

- Immediate configuration updates
- Real-time UI feedback

Key Files Modified

`src/stores/useSchoolItPreviewStore.js`

`src/views/desktop/dashboard/*`

Phase 3: Dark Mode Visibility and Contrast Remediation

Objective

Resolve visibility issues introduced after dark mode deployment.

Several components had low contrast text caused by hardcoded light-mode styling.

Improvements Implemented

Search and Filter Pills

Updated:

- Text color
- Background color
- Hover behavior

Ensured visibility across dark backgrounds.

Aura AI Chat Interface

Refactored chat UI in:

`SchoolItHomeView.vue`

Changes included:

- Removal of hardcoded white backgrounds
 - Dynamic dark mode styling
 - Improved message bubble contrast
-

Form Visibility Enhancements

Refactored:

EventEditorSheet.vue

Improvements:

- Visible input labels
 - Improved placeholder visibility
 - High contrast form controls
-

Key Files Modified

src/views/desktop/dashboard/SchoolItHomeView.vue

src/components/desktop/events/EventEditorSheet.vue

src/views/desktop/dashboard/SchoolItScheduleView.vue

Phase 4: Student Face Verification Security Gate

Objective

Address a critical security issue where student accounts bypassed mandatory face verification after login.

Implementation Details

Generic Verification Gate

Authentication logic was refactored to remove role-specific restrictions.

Files updated:

src/services/localAuth.js

src/composables/useAuth.js

This ensures:

- **Face verification applies to all users**
 - **Security enforcement across roles**
-

Router Security Enhancement

Navigation guards were updated in:

src/router/index.js

New behavior:

- **Sessions with pending face verification are intercepted**
 - **Users are redirected to verification screen**
-

Mock API Alignment

Development API updated:

mock-api.js

Enhancements:

- **Verification flag correctly triggered**
 - **Student accounts require face verification**
-

Role-Aware UI Implementation

Updated verification interface:

PrivilegedFaceVerificationView.vue

Enhancements:

- **Role-based messaging**
 - **Improved UX for students and administrators**
-

Key Files Modified

src/services/localAuth.js

src/composables/useAuth.js

src/router/index.js

src/views/desktop/auth/PrivilegedFaceVerificationView.vue

mock-api.js

Testing Accounts Used

The following accounts were used during system testing and validation:

Super Admin

Email: admin@example.com

Password: password123

Branding: Black / Neutral

Campus 1: Mock University

School IT

Email: it@example.com

Password: password123

Branding: Blue & Yellow

Student

Email: student@example.com

Password: password123

Branding: Blue & Yellow

Campus 2: North Campus College

School IT

Email: it-north@example.com

Password: password123

Branding: Green & White

Student

Email: student-north@example.com

Password: password123

Branding: Green & White

Campus 3: South Institute

School IT

Email: it-south@example.com

Password: password123

Branding: Red & Gold

Summary of Results

UI Improvements

- Consistent Aura dark mode design
 - Premium glass UI styling
 - Improved visual accessibility
-

UX Improvements

- Fully interactive dashboard
 - Real-time updates
 - Improved usability
-

Security Improvements

- Mandatory face verification
 - Role-agnostic authentication
 - Router-level enforcement
-

Reliability Improvements

- **Session-based persistence**
- **Stable development testing**
- **Improved maintainability**

The RIZAL_v1 project has successfully evolved into a modern, secure, and fully interactive system. The enhancements implemented across the four development phases significantly improved user experience, system reliability, and application security.

The project is now ready for further development, testing, and production deployment.

Author

Project Evolution Documentation

RIZAL_v1 Development Team